

Byze: A Peer-to-Peer Electronic Cash System with RandomX Proof-of-Work and Post-Quantum Signatures

Version 0.1.1 — June 2026

Authors: Byze contributors ([powhermes/byze](#))

Contents

1. Abstract
 2. Introduction
 3. Transactions
 4. Timestamp Server and Blocks
 5. Proof-of-Work (RandomX)
 6. Post-Quantum Signatures
 7. Network
 8. Incentive and Monetary Policy
 9. Reclaiming Disk Space
 10. Splitting and Combining Value
 11. Privacy
 12. Security Analysis
 13. Comparison with Bitcoin
 14. Roadmap and Evolution
 15. Conclusion
 16. References
 17. Appendix A — Wire and consensus constants
 18. Appendix B — Example quantum output
 19. Appendix C — Block serialization tail (mainnet)
 20. Appendix D — Mining and pool protocol
-

1.1 Abstract

A purely peer-to-peer version of electronic cash would allow online payments to be sent directly from one party to another without going through a financial institution. Bitcoin demonstrated that such a system is practical when combined with proof-of-work timestamping and public-key signatures. However, Bitcoin's SHA256d mining favors specialized hardware, and its ECDSA/Schnorr signatures are not designed to resist cryptanalytic attacks enabled by large-scale quantum computers.

Byze (BYZ) is a Bitcoin Core–derived electronic cash system that preserves the UTXO model, block chain structure, and economic schedule of Bitcoin while replacing two foundational assumptions: **RandomX** proof-of-work for CPU-friendly mining, and **mandatory dual post-quantum signatures** (XMSS and SPHINCS+) for both transaction authorization and mainnet block attestation. Nodes reach consensus on the same ordered history of transactions without a trusted mint. The system is designed for verifiable, non-custodial participation from commodity hardware.

1. Introduction

Commerce on the Internet has come to rely almost exclusively on financial institutions serving as trusted third parties to process electronic payments. While the system works well enough for most transactions, it still suffers from the inherent weaknesses of the trust-based model. Completely non-reversible transactions are not really possible, mediation costs increase transaction fees, and merchants cannot avoid requesting more information than they would otherwise need.

Cryptographic proof, instead of trust, allows willing parties to transact directly. Bitcoin solved the double-spending problem with a peer-to-peer network that timestamps transactions by hashing them into an ongoing chain of proof-of-work–based records. Byze adopts that architecture wholesale—blocks, Merkle roots, UTXOs, halvings, and fee markets—while hardening the two primitives most exposed to long-term technological shift: **work function** and **digital signatures**.

Byze is **not Bitcoin**. It uses distinct network magic bytes, ports, address prefixes, and genesis parameters. Coins on the Byze chain (BYZ) are not interchangeable with BTC. This document describes the Byze protocol as implemented in the reference client (`byzed`, `byze-qt`, `byze-cli`).

1.1 Design goals

1. **Peer-to-peer electronic cash** — Direct value transfer without custodial intermediaries.
2. **CPU-oriented mining** — RandomX makes commodity processors the intended mining substrate, reducing ASIC centralization pressure relative to SHA256d.
3. **Post-quantum signature hygiene** — Every spend and every mainnet block (after genesis) carries **both** an XMSS-class and a SPHINCS+-class signature over the same message digest, so a break in one family does not silently invalidate the security model.
4. **Bitcoin-familiar economics** — 10-minute mean block interval, 210,000-block halving interval, ~21 million BYZ asymptotic supply cap, non-spendable genesis subsidy.

5. **Operational mainnet** — The network launched with modern script rules (SegWit, Taproot infrastructure) active from height 1, but **only quantum witness v1 outputs are valid for spending**.
-

2. Transactions

Byze follows the Bitcoin transaction model. An **electronic coin** is defined as a chain of digital signatures. Each owner transfers the coin to the next by digitally signing a hash of the previous transaction and the public key of the next owner, and adding these to the end of the coin. A payee can verify the signatures to verify the chain of ownership.

2.1 UTXOs and witnesses

Transactions consume unspent transaction outputs (UTXOs) and create new outputs. Byze uses SegWit serialization (witness data is committed via the witness Merkle root in the block header). Unlike Bitcoin Taproot, which authorizes spends with a single 64-byte Schnorr signature, **Byze quantum outputs require a fixed three-element witness stack**:

Witness element	Size (bytes)	Role
XMSS signature	2,500	Post-quantum stateful hash-based signature (liboqs XMSS-SHA2_10_256)
SPHINCS+ signature	7,856	Stateless post-quantum signature (liboqs SPHINCS+ SHA2-128s-simple)
Dual public-key bundle	100	68-byte XMSS pubkey + 32-byte SPHINCS+ pubkey

The `scriptPubKey` for a standard Byze output is a **witness version 1 program** (the same structural slot Bitcoin uses for Taproot):

```
OP_1 <32-byte program>
```

where:

```
program = SHA256(dual_public_key_bundle)
```

Both signatures must verify against the **BIP341-style transaction sighash** (`SignatureHashSchnorr` code path) of the input being spent. Consensus rejects ECDSA, Schnorr, and witness v0 spends for standard payment paths—only the quantum dual-signature witness shape is accepted in `VerifyScript`.

2.2 Addresses

Human-readable addresses use Bech32 with human-readable part **byz** (mainnet), e.g. `byz1...`. Legacy base58 pay-to-public-key-hash addresses use prefix **H**. These encodings are unique to Byzee and prevent accidental cross-chain payment to Bitcoin addresses.

2.3 Wallet key hierarchy

The reference wallet derives quantum key material deterministically from the same BIP32 extended secret that backs Taproot descriptors. The 32-byte output program is `SHA256(dual_public_key_bundle)`, binding the on-chain program to the published public keys in the witness.

XMSS index discipline: XMSS is a stateful signature scheme—each one-time index must be used at most once. The wallet persists the current index in `wallet.dat`, reserves an index before signing, and advances it atomically after a successful sign. Reusing an index breaks only the XMSS leg (the SPHINCS+ leg remains valid), but consensus still requires both legs; operators must treat index management as a liveness requirement.

3. Timestamp Server and Blocks

The solution to double-spending is a **timestamp server** implemented as a distributed block chain. Each block contains a hash of the previous block header, forming a chain that becomes computationally expensive to rewrite as it grows.

3.1 Block structure

A Byzee block consists of:

1. **Block header** — `nVersion`, `hashPrevBlock`, `hashMerkleRoot`, `nTime`, `nBits`, `nNonce` (unchanged field layout from Bitcoin).
2. **Transaction list** — Ordered transactions including exactly one coinbase.
3. **Quantum signature tail** (mainnet, post-genesis) — Appended to the serialized block after transactions:

```
struct quantum_signature_data {  
    xmss_signature[2500];  
    sphincs_signature[7856];  
    dual_public_key[100];  
};
```

This tail is **not** part of the header hashed for proof-of-work; miners sign `CBlockHeader::GetHash()` after finding a valid RandomX nonce.

3.2 Merkle root and witness commitment

Transaction inclusion follows Bitcoin rules: duplicate txids forbidden, coinbase required, block weight limits enforced. Witness commitments remain in the coinbase per BIP141.

3.3 Compact block relay

BIP152 compact blocks omit the quantum tail by design. Peers that detect a missing tail on mainnet fall back to requesting the full block, ensuring quantum fields propagate correctly across the network.

4. Proof-of-Work (RandomX)

4.1 Motivation

Bitcoin's proof-of-work uses double SHA256 on the block header. This function is extremely cheap on ASICs, concentrating hashrate in specialized datacenters. **RandomX** is a proof-of-work algorithm optimized for general-purpose CPUs: it executes randomized programs in a virtual machine with memory-hard access patterns, making FPGA/ASIC advantage harder to sustain relative to modern x86/ARM cores.

Byze replaces `SHA256(SHA256(header))` with:

```
block_pow_hash = RandomXHash(block_header)
```

A block is valid when `block_pow_hash <= target(nBits)`, using the same compact `nBits` encoding and `powLimit` arithmetic as Bitcoin.

4.2 RandomX lifecycle

The reference node initializes a RandomX cache and dataset at startup. Validation uses a shared global context protected by a mutex. Mining uses **per-thread virtual machines** (`RandomXMiningContext`) so pool software and RPC `generateToAddress` can search nonces in parallel without starving validation.

4.3 Difficulty adjustment

Byze retargets difficulty every **6 blocks** (one hour at the 10-minute target spacing), not Bitcoin's 2016-block window:

Parameter	Byze mainnet
Target spacing	600 s (10 min)
Retarget interval	3,600 s (1 h)
Blocks per retarget	6
Min/max timespan factor	$\frac{1}{4} - 4\times$ (same clamp as Bitcoin)

4.4 Soft launch (bootstrap window)

For the first **10,000 blocks** after genesis, mainnet holds difficulty at `powLimit` regardless of inter-block times. This **soft launch** allows early CPU miners to find blocks without an initially extreme hashrate requirement. After height 10,000, normal retargeting applies.

4.5 Genesis

Mainnet genesis (RandomX-mined):

Field	Value
Timestamp message	Byze (BYZ) genesis - RandomX PoW - 2026-04-27
Subsidy	50 BYZ (non-spendable, as in Bitcoin)
Block hash	00000a6ff06ac3c6549d30a1931c8ff1fc98be705c7802164...
Merkle root	946448a64c948d5a9c044306689cc47a4e1c12ee591f8871...

The genesis block is **exempt** from the quantum block-signature requirement.

5. Post-Quantum Signatures

5.1 Threat model

Shor's algorithm breaks discrete-log and factoring-based public-key cryptography on a sufficiently large fault-tolerant quantum computer. While the timeline for such machines is uncertain, a blockchain intended to operate for decades must plan for **Harvest Now, Decrypt Later** attacks against long-lived UTXOs and for the eventual need to migrate signature schemes without a trusted coordinator.

Byze addresses this at the protocol layer by requiring **two independent post-quantum signature families** on every spend and on every mainnet block.

5.2 XMSS (stateful hash-based signatures)

XMSS (eXtended Merkle Signature Scheme, standardized as RFC 8391) is a stateful hash-based signature scheme. Security reduces to collision resistance of the chosen hash function rather than hardness of discrete logarithms. The trade-off is **state**: each key has a finite number of one-time slots (Byze defaults: tree height **10** → **1,024** signatures per key).

Byze XMSS parameters (mainnet, implemented via `liboqs XMSS-SHA2_10_256`):

Parameter	Value
Tree height	10
Max signatures per key	1,024
Signature wire size	2,500 bytes

XMSS pubkey size	68 bytes in the dual bundle
SPHINCS+ pubkey size	32 bytes in the dual bundle
Dual bundle total	100 bytes (68 + 32)

The reference implementation uses `liboqs OQS_SIG_STFL_alg_xmss_sha256_h10` for signing and verification, with HKDF-based deterministic key derivation from wallet entropy (`byze/q/xmss_seed` label). Future releases may adopt alternate RFC 8391 parameter sets without changing the outer witness envelope layout.

5.3 SPHINCS+ (*stateless hash-based signatures*)

SPHINCS+ (standardized as FIPS 205 / SLH-DSA) provides stateless post-quantum signatures at the cost of larger signatures. Byze uses it as a **second line of defense**: wallets and miners must produce a valid SPHINCS+ signature over the same digest even if XMSS state management fails or XMSS is cryptanalyzed.

Byze SPHINCS+ parameters (mainnet, implemented via `liboqs SPHINCS+-SHA2-128s-simple`):

Parameter	Value
Security level	1 (<code>liboqs sphincs_sha2_128s_simple</code>)
Signature wire size	7,856 bytes
SPHINCS+ pubkey size	32 bytes

As with XMSS, key derivation uses HKDF from wallet entropy (`byze/q/sph_seed_a` label). The protocol slot is reserved for standards-aligned parameter upgrades.

5.4 Dual-signature rule

For **transactions**, both signatures must verify against the sighash and the dual public-key bundle must hash to the output program.

For **blocks** (mainnet, height ≥ 1):

1. `xmss_signature`, `sphincs_signature`, and `dual_public_key` must be present with exact expected sizes.
2. Both signatures must verify over `CBlockHeader::GetHash()`.
3. Partial tails (only one signature present) are rejected.

Blocks missing the tail on mainnet are invalid with consensus error `bad-quantum-sig-missing`. Invalid cryptography yields `bad-quantum-sig`.

Test networks may omit block tails unless `-enforcequantumblocksigns` is set; if any quantum field is present, all fields must be valid.

5.5 Why dual schemes?

Concern	Mitigation
Quantum break of ECDSA/Schnorr	Neither is used for Byze payments
XMSS state loss / index reuse	Operator tooling + SPHINCS+ leg still required; wallet persists index in DB
Break of one PQ family	Second family still required at consensus
Large signatures	Acceptable trade-off for base-layer security; witness discount applies

6. Network

6.1 Peer discovery and isolation

Byze nodes speak a Bitcoin-derived P2P protocol with distinct **message start bytes** `0x48 0x45 0x52 0x4D` ("HERM") so clients never accidentally connect to Bitcoin mainnet. Default mainnet port: **8888**.

6.2 Transaction relay policy

Standard transaction policy (`IsStandard`) accepts only:

- `witness_v1_taproot` quantum outputs, and
- `OP_RETURN` data carriers within size limits.

Bare multisig, P2PKH, P2SH-wrapped legacy scripts, and witness v0 outputs are non-standard and not relayed by default nodes. This nudges the ecosystem toward quantum outputs while keeping consensus rules explicit in the script interpreter.

6.3 Mining integration

Solo mining: RPC `startmining`, `generatetoaddress`, and the in-process controller search RandomX nonces, then attach quantum block signatures automatically before submission.

Pool mining: External miners using `getblocktemplate` / `submitblock` must ensure submitted blocks include the quantum tail. Pool operators typically call the `signpoolblock` RPC on a node holding the pool's quantum block-signing keys to attach XMSS + SPHINCS+ signatures after a valid RandomX nonce is found. Stratum reference deployments use `byze-miner` against pools (e.g. `pool.byze.org:3333`).

See **Appendix D** for the full pool and block-submission protocol.

6.4 Simplified Payment Verification (SPV)

Light clients can follow the longest proof-of-work chain by downloading block headers and Merkle branches, as in Bitcoin. **Additional caveat:** SPV clients that do not download full blocks cannot verify quantum block signatures in the tail; they rely on full nodes having rejected invalid tails. For highest assurance, run a full validating node (byzed).

7. Incentive and Monetary Policy

Byze inherits Bitcoin's issuance schedule:

Rule	Value
Initial subsidy	50 BYZ per block
Halving interval	210,000 blocks (~4 years at 10 min/block)
Minimum relay fee	Node policy (default Bitcoin-derived settings)
Total supply cap	Approaches 21,000,000 BYZ
Spendable supply	~20,999,950 BYZ (genesis coinbase unspendable)

Fees: transaction inputs minus outputs, same as Bitcoin. As subsidies halve toward zero, incentive shifts to fees and block space markets.

The **shorter difficulty window** (6 blocks vs 2016) makes Byze more responsive to hashrate changes but increases variance in retarget noise during the first hours of each window. This is an intentional deviation for a young CPU-mined network.

8. Reclaiming Disk Space

Once a coin's latest spend is buried under sufficient work, the spend can be pruned from full-node storage if Merkle commitments allow—identical considerations as Bitcoin. Byze's larger witnesses (~**10 KB quantum material per input** vs ~64 bytes Schnorr) increase bandwidth and storage growth relative to Bitcoin Taproot; pruned nodes and `assumeUTXO` snapshots remain supported in the codebase for operational scaling.

9. Splitting and Combining Value

Transactions may contain multiple inputs and outputs. Typical quantum spends consume one or more UTXOs and produce change outputs using the same witness v1 quantum program format. Coin selection, feerate estimation, and RBF policies follow Bitcoin Core wallet logic adapted for larger witness weight.

Weight note: Each quantum input contributes roughly **10 KB** of witness data (2,500 B XMSS + 7,856 B SPHINCS+ + 100 B dual pubkey). Users should expect higher minimum fees for small payments than on Bitcoin Taproot.

10. Privacy

Byze inherits Bitcoin's pseudonymous model: public keys (here, hashed dual bundles) are not inherently linked to real-world identity, but the graph of transactions is public. Quantum outputs reuse Taproot's single 32-byte program appearance on-chain, but witnesses are substantially larger (~10 KB) and more distinctive in mempool traffic analysis.

Recommended practices:

- Use fresh receive indices for privacy-sensitive receipts.
- Do not reuse XMSS indices across wallets.
- Run your own node; third-party explorers see the same public chain data.

Byze does not implement protocol-layer confidential transactions.

11. Security Analysis

11.1 Security properties

Property	Definition	Mechanism
Ledger integrity	Only valid spends of existing UTXOs enter the canonical chain	UTXO model + script verification
Chain ordering	History cannot be rewritten without majority work	RandomX cumulative proof-of-work
Authorization binding	Only holders of quantum private state can move value	Dual XMSS + SPHINCS+ witnesses
Block authenticity	Mainnet blocks commit to a known signing identity	Quantum tail over header hash
Network partition safety	Conflicting tips resolve to the most-work chain	Longest (most-work) chain rule

11.2 Threat model

We consider an adversary that can:

- Control a fraction (η) of total RandomX hashrate.
- Monitor and censor network traffic (delay, reorder, drop messages).

- Run a large-scale quantum computer capable of Shor's algorithm on ECDLP—but **not** capable of breaking SHA256/SHA3 collision resistance at 256-bit security (standard conservative assumption for hash-based PQ schemes).
- Compromise individual users' wallets or pool operators' signing nodes (out of scope for base-layer consensus; mitigated operationally).

We do **not** assume trusted third parties, hardware security modules, or honest majority of stake—only work, as in Bitcoin.

11.3 Proof-of-work security

Let honest hashrate be $((1-\eta)H)$ and adversarial hashrate be (ηH) . The probability that an attacker with $(\eta < 0.5)$ catches up from (z) blocks behind follows the same birthday/random-walk analysis as in Bitcoin's original paper: as (z) increases, success probability drops exponentially.

RandomX-specific considerations:

- **Hardware parity:** RandomX is designed so CPU implementations are competitive; an ASIC advantage may exist but is expected to be smaller than for SHA256d. This improves participation fairness but does not remove the 51% threshold.
- **Soft launch:** During heights $(1 \dots 10,000)$, difficulty is fixed at `powLimit`, temporarily lowering the work cost of reorgs. Applications requiring strong finality during bootstrap should wait for sufficient depth and passage of the soft-launch window.
- **Fast retarget:** Six-block windows react quickly to hashrate influx; operators should expect difficulty swings during the network's early life.

Proposition (51% reorg): An adversary with $(\eta > 0.5)$ can, given sufficient time, produce a longer valid chain and reverse confirmations. No proof-of-work chain eliminates this; Byzantine additionally requires valid quantum block tails on the fork, so the attacker must also possess the pool/node block-signing keys—or mine solo with a wallet that signs blocks.

11.4 Post-quantum signature security

Transaction forgery. To forge a spend of output program (P) , an adversary must produce witness (σ_x, σ_s, K) such that:

1. $(\text{SHA256}(K) = P)$
2. $(\text{VerifyXMSS}(\text{sighash}, \sigma_x, K_x) = \text{true})$
3. $(\text{VerifySPHINCS}+(\text{sighash}, \sigma_s, K_s) = \text{true})$

where $((K_x, K_s))$ partition the **100-byte bundle** (K) into the 68-byte XMSS pubkey and 32-byte SPHINCS+ pubkey.

Under **independence** of the two verification functions and **collision resistance** of the hash functions used internally, forging without the private state requires breaking at least one

scheme. A break of ECDSA/Schnorr alone is insufficient because those key types are not consensus-valid for payments.

11.5 Mempool and relay attacks

Because quantum witnesses are large (~10 KB per input), denial-of-service via oversized transactions is mitigated by standard weight limits (`MAX_BLOCK_WEIGHT`, `standard policy`). Non-quantum outputs are non-standard and not relayed, reducing legacy malleability vectors but concentrating traffic shape on quantum witnesses.

11.6 Operational risks

Risk	Consequence	Mitigation
XMSS index reuse	Transaction rejected	Wallet DB persistence, pending-index recovery
Loss of wallet.dat	Loss of funds	Backups, optional BIP39 recovery phrase in Qt wallet
Large witnesses	Higher fees	Batch payments, appropriate feerates
Pool omitting block tail	Block rejected	Call <code>signpoolblock</code> before <code>submitblock</code>
Compromised pool signer	Attacker can sign invalid blocks with valid PoW	Key rotation, multi-party block signing (future)
SPV-only clients	Cannot verify quantum block tails	Run full node for full assurance

11.7 Limitations and open problems

- Block signing keys are node-local; decentralized block-signer rotation is not yet specified.
- Quantum witness size increases layer-1 bandwidth costs versus Bitcoin Taproot (~10 KB per input vs ~64 bytes).
- RandomX performance on non-x86 architectures varies; pool software should benchmark per platform.
- Migration path to standards-track XMSS (RFC 8391) and SPHINCS+ (FIPS 205) byte layouts is planned for a future protocol version.

12. Comparison with Bitcoin

Aspect	Bitcoin	Byze
PoW hash	SHA256d	RandomX
Payment signatures	ECDSA / Schnorr	XMSS + SPHINCS+ (dual, mandatory)

Block signatures	None (header only)	Dual PQ tail on mainnet
Standard outputs	P2WPKH, P2TR, etc.	Quantum witness v1 only (policy)
Retarget period	2016 blocks (~2 weeks)	6 blocks (~1 hour)
Soft launch	None	10,000 blocks min difficulty
Witness size (per input)	~64 bytes (Taproot)	~10 KB (2500 + 7856 + 100)
Network magic	0xF9 0xBE 0xB4 0xD9	0x48 0x45 0x52 0x4D
Default P2P port	8333	8888
Bech32 HRP	bc / tb	byz
Ticker	BTC	BYZ

Byze is a **hard fork in spirit**—a new genesis chain—not a live Bitcoin upgrade path.

13. Roadmap and Evolution

The live mainnet implements RandomX PoW, quantum transaction witnesses, and quantum block tails. Near-term engineering focuses on:

- **Distribution** — Signed desktop bundles (Linux, Windows, macOS), reproducible builds.
- **Standards alignment** — Migrating internal XMSS/SPHINCS+ engines toward RFC 8391 / FIPS 205 byte layouts without breaking the dual-witness envelope.
- **Ecosystem** — Explorer, browser wallet, public pool, and reference CPU miner (`byze-miner`).
- **Hardware workflows** — Integration with external signers and backup tooling as the network matures.

Protocol upgrades follow Bitcoin-style soft-fork discipline where applicable; witness version 1 quantum layout occupies the Taproot upgrade lane, leaving higher witness versions available for future extensions.

14. Conclusion

Byze demonstrates that the Bitcoin architecture—UTXO ledger, proof-of-work ordering, Merkle commitments, and halving economics—can be extended to address two long-horizon threats: **mining hardware centralization** and **quantum cryptanalysis**. By substituting RandomX for SHA256d and requiring dual post-quantum signatures at both the transaction and block layers, Byze offers a path toward CPU-accessible mining and signature agility without abandoning the peer-to-peer cash model.

We proposed a system for electronic transactions without relying on trusted third parties or pre-quantum signature assumptions at the spending layer. Nodes vote with their CPU power on the RandomX chain, rejecting invalid work and invalid quantum proofs. Rules and incentives are enforced by consensus code open to public audit.

References

1. S. Nakamoto, Bitcoin: A Peer-to-Peer Electronic Cash System, 2008. <https://bitcoin.org/bitcoin.pdf>
 2. tevador et al., RandomX: A CPU-friendly proof of work, 2019.
 3. B. Buchmann et al., XMSS: Extended Hash-Based Signatures (RFC 8391).
 4. J. Bernstein et al., SPHINCS+ (NIST PQC; FIPS 205 / SLH-DSA).
 5. BIP141 — Segregated Witness; BIP340–342 — Schnorr/Taproot (Byze reuses sighash machinery).
 6. Byze source: <https://github.com/powhermes/byze>
 7. Network hub: <https://byze.org>
 8. Open Quantum Safe (liboqs): <https://openquantumsafe.org>
-

Appendix A — Wire and consensus constants

Source of truth: `src/crypto/quantum_safe_config.h`

```
BYZE_XMSS_SIGNATURE_SIZE      = 2500    //
OQS_SIG_STFL_alg_xmss_sha256_h10_length_signature
BYZE_SPHINCS_SIGNATURE_SIZE   = 7856    //
OQS_SIG_sphincs_sha2_128s_simple_length_signature
BYZE_XMSS_PUBKEY_SIZE         = 68      //
OQS_SIG_STFL_alg_xmss_sha256_h10_length_pk
BYZE_SPHINCS_PUBKEY_SIZE      = 32      //
OQS_SIG_sphincs_sha2_128s_simple_length_public_key
BYZE_DUAL_PUBKEY_BUNDLE_SIZE  = 100     // XMSS pubkey + SPHINCS+ pubkey
BYZE_DEFAULT_XMSS_TREE_HEIGHT = 10
BYZE_DEFAULT_SPHINCS_LEVEL    = 1       // sphincs_sha2_128s_simple
WITNESS_V1_TAPROOT_SIZE      = 32
SOFT_LAUNCH_HEIGHT            = 10000
SUBSIDY_HALVING_INTERVAL      = 210000
POW_TARGET_SPACING             = 600 seconds
POW_TARGET_TIMESPAN           = 3600 seconds
```

Appendix B — Example quantum output

scriptPubKey: `OP_1 0x<32-byte SHA256(dual_public_key_bundle)>`

scriptWitness.stack:

```
[0] xmss_signature      (2500 bytes)
[1] sphincs_signature   (7856 bytes)
[2] dual_public_key     ( 100 bytes)
    └─ xmss_pubkey      [0:68]   (68 bytes)
    └─ sphincs_pubkey   [68:100] (32 bytes)
```

Verification (consensus):

- 1. SHA256(stack[2]) == program in scriptPubKey
- 2. VerifyXMSS(sighash, stack[0], pubkey_xmss) succeeds
- 3. VerifySPHINCS+(sighash, stack[1], pubkey_sphincs) succeeds

where pubkey_xmss = stack[2][0:68] and pubkey_sphincs = stack[2][68:100].

Appendix C — Block serialization tail (mainnet)

Full block serialization order:

```
CBlockHeader
var-int tx count
transactions[]
quantum_signature_data  // required on mainnet except genesis
    xmss_signature      [2500 bytes]
    sphincs_signature   [7856 bytes]
    dual_public_key     [ 100 bytes]
```

Block proof-of-work is validated on the header alone; quantum signatures bind the miner's keys to that header hash after PoW is found.

Appendix D — Mining and pool protocol

This appendix specifies how miners, pools, and node operators interact with Byze mainnet. It is descriptive of the reference implementation (byzed v0.1.0).

D.1 Roles

Role	Software	Responsibility
Full node	byzed	Validates chain, serves GBT, signs blocks (signpoolblock), submits blocks
Solo miner	byzed + RPC or Qt	Finds RandomX nonce, auto-signs quantum tail, broadcasts block
Pool server	Custom + byzed	Issues work units, aggregates shares, builds candidate blocks

Pool miner	byze-miner	Computes RandomX hashes for pool-assigned header/nonces
Payout wallet	Any quantum wallet	Receives coinbase to a byz1... address

D.2 Stratum mining (recommended for pool miners)

The public reference pool listens on **pool.byze.org:3333** (Stratum). Miners run the standalone RandomX client:

```
byze-miner/build/byze-miner \
  --pool pool.byze.org:3333 \
  --wallet byz1YOUR_PAYOUT_ADDRESS \
  --worker rig01 \
  --threads 8
```

D.3 Solo mining via RPC

A node with a loaded wallet and quantum block-signing keys can mine in-process:

```
byze-cli startmining "byz1YOUR_ADDRESS" 8
```

Or generate a fixed number of blocks (functional tests, regtest):

```
byze-cli generatetoaddress 1 "byz1YOUR_ADDRESS"
```

The node searches RandomX nonces with per-thread VMs, attaches the quantum tail via `MaybeSignBlockQuantum`, and submits the block to the local chain.

D.4 Pool block pipeline (GBT + sign + submit)

```
Pool controller          byzed (full node)
  |   getblocktemplate   |
  | ----->|
  | <-----| template + jobs
  |
byze-miner workers ---> Pool validates RandomX shares
                        |
                        winning header+nonce
                        v
                        signpoolblock (byze-cli)
                        |
                        hex + quantum tail
                        v
                        submitblock
```

Step 1 — Template: `getblocktemplate` with `{"rules":["segwit"]}`.

Step 2 — PoW search: Miners vary `nNonce` until `RandomXHash(header) <= target(bits)`.

Step 3 — Sign: Call `byze-cli signpoolblock "<hex>"`. The node attaches `xmss_signature`, `sphincs_signature`, and `dual_public_key` over `CBlockHeader::GetHash()`.

Step 4 — Submit: `byze-cli submitblock "<hex from signpoolblock>"`. Blocks missing the quantum tail on mainnet are rejected with `bad-quantum-sig-missing`.

D.5 Coinbase and payouts

Coinbase value equals subsidy at height (`h`) plus fees: $V_{\text{coinbase}} = \text{GetBlockSubsidy}(h) + \Sigma \text{fees}$. Use a quantum `byz1...` payout address so rewards are spendable with standard wallets.

D.6 Checklist for pool operators

1. Run a synced mainnet byzed with `-txindex` if your pool software requires it.
2. Ensure the node wallet holds quantum block-signing keys (solo mining once or import pool signer material).
3. After any valid PoW, always call **signpoolblock** before **submitblock** on mainnet.
4. Monitor `quantum_xmss_index` via wallet RPC—block signing consumes XMSS state.
5. Backup `wallet.dat` for the signing wallet; loss equals inability to produce valid blocks.
6. Set firewall rules: RPC only on localhost; expose Stratum port publicly.

This document describes the Byzee protocol as of release v0.1.1. Implementations should treat the reference source code ([powhermes/byze](https://github.com/powhermes/byze)) and consensus tests as authoritative if any discrepancy arises.